Project Research Notebook

ACE

CYBER R&D Program

PI: Teaundra Thompson

University: Charleston Southern University

Project Title: IDS Resiliency Against Evasion Attacks

1. Purpose and Best Practices:

This document is the official chronological log of all experiments, observations, and manual actions performed within the ACE project testbed. It is the primary source of "ground truth" for what occurred during the research. It is a living document.

Best Practices:

- **Chronological Order:** New entries must always be added to the end of the log.
- **Be Verbose:** Write more than you think you need. Future you will be grateful. Capture thoughts, hunches, and qualitative observations.
- **Timestamp Everything:** At a minimum, capture the date in the format mm/dd/yyyy. If time is relevant, capture the timestamp in UTC to avoid confusion and simplify log correlation across different systems.
- **Record Failures:** Failed experiments are data. Document what went wrong, why you think it went wrong, and what you did to fix it.
- **Link, Don't Duplicate:** Reference external documents (like Test Plans) by their ID, version, and step. Do not paste their contents here.
- **Immutable Raw Data:** Note the location and checksums of raw data. The raw data itself should never be modified. Analysis is performed on copies.

2. Experiment Log

This section will capture each test conducted in your research project testbed. Each execution of your test will be denoted with a “Execution ID”. Use the template provided for the first two entries. Additional entries can be made by using the copy and past from the template at the end of this document. Adjust the template as necessary to support your project. **Important:** Be detailed as this will help you conduct your analysis, your report, and provide transparency with reproducibility by others.

Execution ID: RUN-20251124-001

Date: 2025-11-24

Start Time (UTC): 17:52:13

End Time (UTC): 18:17:27

Researcher(s): Teaundra Thompson

A. Linked Documents:

Test Plan: TP-001-v1.4

Test Case(s): TC-001

B. Objective:

Ensure no false positives arise when a pcap with normal traffic is inserted into IDS. Also ensure that the MLP model can understand Zeek's conn.log.

C. Summary of Actions:

Note that the pcap was created during the test case execution. The VM's Network adapter was switched back to NAT, it was previously on Bridged Adapter due to network connectivity issues with CSU_Wireless. For the creation of pcap file "normalTraffic.pcapng", Wireshark was opened and prepared to begin capturing packets on enp0s3. I then engaged in typical web browsing utilizing Firefox within the VM enviroment. After three minutes of web browsing I stopped the packet capture and saved the pcap file as in the TC1 folder. For the actual execution of the test case, the VM snapshot "Begin Test Cases" was powered on from a save state. The terminal was opened and command "cd Desktop/testCases/TC1" was executed to navigate to test case one's folder. The pcap was then fed into Zeek by execution of the command "zeek -r normalTraffic.pcapng", but Zeek gave an warning that checksum offloading was occuring so I deleted the generated logs and ran "zeek -C -r normalTraffic.pcapng" to analyze the checksum, but that did not work. I deleted the pcap and recreated it twice to see if there was any change, but it was no avail. I ran the command "zeek -r normalTraffic.pcapng" on the third created pcap and allowed the checksum offloading. The third created pcap has the name as the first pcap, "normalTraffic.pcapng", I proceeded the test case with this pcap. The checksum offloading most likely came from the websites I was visiting and the ads that were playing. Then Zeek generated the conn.log into the TC1 folder. FireFox was opened and I navigated my Colab environment to the MLP notebook. The UFW-Zeek24 dataset files were uploaded to retrain the model. After training, the model and the preprocessors were saved and downloaded to file path /home/tootie/Desktop/testCases/MLP Model to be reuploaded as needed to the Colab to complete other test cases. The conn.log file was uploaded to the notebook environment and was preprocessed and then analyzed.

D. Key Observations (Qualitative Data):

It seems as if websites with copious amounts of ads cause pcap captures to have checksum errors, compared to websites with minimal ads.

E. Anomalies, Errors, and Deviations from Test Plan:

I remade the normalTraffic.pcapng twice to try and prevent checksum offloading and checksum errors, but it did not work. The reason I remade the pcaps is because when the MLP was trained on the NSL-KDD set I assumed that the checksum offloading/errors were causing inaccurate analysis, but that is not true. When I allowed the checksum offloading to occur on the third created pcap, the MLP model analyzed the conn.log correctly. The desired output was achieved, the MLP flagged the conn.log as normal. Test case passed.

F. Testbed State Changes:

None

G. Data Pointers:

Raw Data Location: /home/tootie/Desktop/testCases/TC1

Data Integrity Checksums (SHA256):

8c8396ed132855d856cfb067ae97a286d398f8d51347b4e16f3ed2d7937cd1b9 conn.log

7e84409f694767167c5af526589c5ead72af8f7b1f52982082ba465a9282f6d6 conn_log_predictions_analysis.png

c1acd9cc565bf4fc2ea37cf0c67b71646d2ee2dbc697f354f656ad1615de3193 normalTraffic.pcapng

c152ffdbf02bf895f35480ae463eecacfacc065a885ce5bb1cf17880a81e336f TC1_Results.png

H. Next Steps / Follow-up Actions:

Proceed with TC2

Execution ID: RUN-20251124-001

Date: 2025-11-24

Start Time (UTC): 18:55:7

End Time (UTC): 19:08:43

Researcher(s): Teaundra Thompson

A. Linked Documents:

Test Plan: TP-001-v1.4

Test Case(s): TC-002

B. Objective:

Observe model behavior when exposes to DDoS/SYN flood traffic. Assess anomaly detection sensitivity when the true MITRE tactic, Impact, is absent from the training dataset. **Test case is exploratory.**

C. Summary of Actions:

Beforehand the pcap file for this test case was created. For the creation of pcap file "SYNFlood.pcapng", I used an additional VM, Kali Linux for the target. The Kali Linux machine was booted up and both Kali Linux and the Zeek VM was switched from the NAT network adapter to the Host- only adapter. This was to prevent the DDoS attack from affecting other devices on the network, and keep it contained within the host machine. Wireshark was opened on the Kali Linux machine to verify that it was receiving packets. Wireshark was opened and prepared to begin capturing packets on enp0s3. I then opened the terminal and ran the command "**sudo hping3 -S –flood –rand-source -p 80 192.168.56.102**". I utilized the command **ctrl+c** after 1.5 minutes to kill the command and to prevent the host machine from crashing. I then stopped the packet capture and saved the pcap file in the TC2 folder. As SYNflood.pcapng. The pcap capture on Kali Linux was discarded. For the actual execution of the test case, the terminal was opened and command **cd..** then **cd TC2** was executed to navigate to test case two's folder (Completing test case two in the same session as TC1). The pcap was then fed into Zeek by execution of the command **zeek -r SYNflood.pcapng**. Then Zeek generated the conn.log into the TC2 folder. Firefox was opened and I navigated my Colab environment to the MLP notebook. I deleted TC1 conn.log out of the Colab runtime and uploaded the conn.log for TC2. The conn.log file was uploaded to the notebook environment and was preprocessed and then analyzed.

D. Key Observations (Qualitative Data):

Due to the size of the pcap, (1,286,500 packets), it took nearly a minute for Zeek to process it. Then it took over a minute for the MLP to preprocess the conn.log.

E. Anomalies, Errors, and Deviations from Test Plan:

1. Model behavior when exposed to DDoS traffic

    - Flagged 100.00% of the traffic as anomalous

2. Analysis of traffic

        - MLP classified the event as a persistence tactic

F. Testbed State Changes:

Added output for most frequent anomaly type in analysis results cell block.

G. Data Pointers:

Raw Data Location: /home/tootie/Desktop/testCases/TC2

Data Integrity Checksums (SHA256):

773e9fa3b01a59ac4b6542a9642041ec8982ede9d2759fcf9cd03e0638db300c conn.log

8c288fda06249e3f746f676873e51cb04c0e8c1f248b3eaae19ef46dd728d9fa
conn_log_predictions_analysis.png

38c3f3e9db4ff9cd9861f2c524079635eadf0e6bbe330614e0a70d1b20819d2a
SYNflood.pcapng

2a517863954fae893eafc56ed5f25dd6f5c8eac15b99fe5f81a0c5ebbfbefb2a
TC2_results.png

H. Next Steps / Follow-up Actions:

Proceed with TC3

Execution ID: RUN-20251124-001

Date: 2025-11-24

Start Time (UTC): 20:25:21

End Time (UTC): 20:39:52

Researcher(s): Teaundra Thompson

A. Linked Documents:

Test Plan: TP-001-v1.4

Test Case(s): TC-003

B. Objective:

Determine the accuracy of the IDS when faced with typical evasion attacks. **Test case is Pass/Fail**

C. Summary of Actions:

Before beginning the test case the script for generating the pcap was created utilizing Claude. The script utilizes Scapy, and creates evasive traffic that uses overlapping fragmentation to evade detection. I navigated to **Desktop/testCases/TC3** by running the commands **cd ..** and **cd TC3**. I opened Wireshark and had it ready to capture packets on the enp0s3 interface. Ran overlapFragment.py script with command **sudo python3 overlapFragment.py**. I engaged in light web browsing while the command was running. When the command stopped after three minutes, I saved the pcap as **fragmentation.pcapng** in **testCases/TC3**. I then ran command **zeek -r fragmentation.pcapng**, then Zeek generated the conn.log into the TC3 folder. My Colab environment was still open. I deleted TC2 conn.log out of the Colab runtime and uploaded the conn.log for TC3. I then ran the cell blocks to preprocess, analyze, and create a graph for the conn.log. After getting results the Colab runtime was disconnected.

D. Key Observations (Qualitative Data):

The MLP processed the conn.log rather fast due to its smaller size.

E. Anomalies, Errors, and Deviations from Test Plan:

Test case passed

F. Testbed State Changes:

None

G. Data Pointers:

Raw Data Location: /home/tootie/Desktop/testCases/TC3

Data Integrity Checksums (SHA256):

eac79fde81a834643a2807c42124dcec1716b8de776fec9c6daac2403c065f0f conn.log

76ab99b027cb918568380c77f8a9ed16607f2536258a36af8db258c47e27f066
conn_log_predictions_analysis.png

13b34600a346decf081ebec38c7139145afc9cb8232d38d4e4603d526f6ee9ca
fragmentation.pcapng

d043863658293eb436d5c78d9c021f9ac608c28c506f484a8d0737e6ce76a250
overlapFragment.py

2b03fb954f471dab3c3e8544d4cef12b0d11dd02b5d2adc2bc8f0c78879e5303
TC3_Result.png

H. Next Steps / Follow-up Actions:

Take snapshot 2 and shutdown VM and Proceed with TC4.

Execution ID: RUN-20251126-001 (e.g., RUN-20231027-001)

Date: 2025-11-26

Start Time (UTC): 12:36:31

End Time (UTC): 12:42:25

Researcher(s): Teaundra Thompson

A. Linked Documents:

Test Plan: TP-001-v1.4

Test Case(s): TC-004

B. Objective:

Determine if the IDS can detect malicious TCP connections made by spyware. This malware exhibits Defense Evasion, Discovery, and Execution tactics which the MLP model was not trained on. **Test case is exploratory.**

C. Summary of Actions:

The pcap (**spyware.pcapng**) was procured from Recorded Future Triage and stored in folder **TC4**. I opened the terminal and ran the command **cd Desktop/testCases/TC4.** The pcap was fed into Zeek using the command **zeek -r spyware.pcapng.** The conn.log was

generated in the TC4 folder and was then uploaded into the Colab environment along with the previously saved model and preprocessors. The model and preprocessors were loaded then the conn.log was analyzed and a graph was created.

D. Key Observations (Qualitative Data):

Everything appeared to run smoothly during this test case.

E. Anomalies, Errors, and Deviations from Test Plan:

1. Model behavior when exposed to discreet spyware traffic

    - Flagged 23.91% of the traffic as anomalous

2. Analysis of traffic

        - MLP classified the traffic as normal

F. Testbed State Changes:

None

G. Data Pointers:

Raw Data Location: /home/tootie/Desktop/testCases/TC4

Data Integrity Checksums (SHA256):

416b465df1d0e3c0a40a0aa7f0b0ba5de5c11116b7f1f765a1812fa48e941c9d conn.log

3cd32aa0f03cd71fcdec52f3f628c0eb504ff89dfa09401263f929296a789d38 conn_log_predictions_analysis.png

abacb7a9444a4c513df97fdb01cbcf83400a0537fdaa33f1c0375781aad27e1b spyware.pcapng

fed62e154832f389c0d67ac448d2cfeb5ca4964995ff2a34958d46d316cba0f8 TC4_Results.png

H. Next Steps / Follow-up Actions:

Proceed with TC-005.

Execution ID: RUN-20252226-SEQ (e.g., RUN-20231027-001)

Date: 2025-11-26

Start Time (UTC): 13:43:05

End Time (UTC): 13:49:32

Researcher(s): Teaundra Thompson

A. Linked Documents:

Test Plan: TP-001-v1.4

Test Case(s): TC-005

B. Objective:

Determine if the MLP can detect password cracking attempts that are spread over a period of time in an evasive manner. Password cracking falls under the Credential Access tactic, which the MLP is trained on. **This test case is exploratory.**

C. Summary of Actions:

This run was completed in the same runtime session as TC4. I started a new capture in Wireshark on enpos3. I opened the terminal on the Zeek VM and ran the command **sudo hping3 -p 22 -i 2 10.0.2.2** to target the SSH on the host machine. This command sent a packet to port 22 every 2 seconds. While that command was running I engaged in light webbrowsing on Firefox for three minutes. After three minutes, I killed the command and saved the pcap capture as **bruteForce.pcapng** in folder TC5. I then ran the commands **cd ..** and **cd TC5** to change the directory to test case five's folder. Then I ran the command **zeek -r bruteForce.pcapng** to feed the pcap into Zeek. The conn.log was generated in the TC5 folder, and was then uploaded to the Colab environment, preprocessed and analyzed.

D. Key Observations (Qualitative Data):

Realized that the training dataset actually does not display brute force/password cracking attempts to port 22, only to port 443. Therefore, the MLP will not flag traffic to port 22 as credential access.

E. Anomalies, Errors, and Deviations from Test Plan:

1. Model behavior when exposed to evasive password cracking

    - Flagged 29.5% of the traffic as anomalous

2. Analysis of traffic

        - MLP classified the traffic as normal

F. Testbed State Changes:

None

G. Data Pointers:

Raw Data Location: /home/tootie/Desktop/testCases/TC5

Data Integrity Checksums (SHA256):

4a5607fe6fec3955a8a66fab00441e67168114add652175340d849d4b66684b5 bruteForce.pcapng

42beabbf79e6521930bdd7d31a4e12c2c1dbef02de063b904becfe98bfff136b conn.log

ec4e320621ba1e81be11cb3b85b5481df45eb2f9162a0fb1d44038a4795b80a6 conn_log_predictions_analysis.png

be3378a46b59c36207c36044940d9eb9ab4c8970e8471d182404884b7c70de6c TC5_Results.png

H. Next Steps / Follow-up Actions:

Proceed with TC6.

Execution ID: RUN-2025126-001 (e.g., RUN-20231027-001)

Date: 2025-12-06

Start Time (UTC): 12:34:33

End Time (UTC): 12:52:17

Researcher(s): Teaundra Thompson

A. Linked Documents:

Test Plan: TP-001-v1.5

Test Case(s): TC-006

B. Objective:

Determine if the MLP can detect port scanning ; consider the fact the MLP was trained on the Reconnaissance tactic. **Test case is pass/fail**

C. Summary of Actions:

VM was powered on from save state and I navigated to "https://github.com/Ryan-Sapone/Nmap-and-Wireshark-Lab/blob/main/SS%20Scan.pcapng" to download the **SS_Scan.pcapg** file. When the pcap was downloaded and moved to the TC6 folder I ran the commands **cd Desktop/testCases/TC6** and **zeek -r SS_Scan.pcapng.** The conn.log was generated and then it was uploaded to the Colab environment along with the previously saved model and preprocessors.

D. Key Observations (Qualitative Data):

The pcap and conn.log was analyzed rather quickly due to the small sizes.

E. Anomalies, Errors, and Deviations from Test Plan:

Test Case passed

F. Testbed State Changes:

None

G. Data Pointers:

Raw Data Location: /home/tootie/Desktop/testCases/TC6

Data Integrity Checksums (SHA256):

da59b438f02256cac2bb0e6d06d50c7319c9512fb96b452d98272713f19888dc conn.log

afa9fe626ec6c3157d7bd500b941e5657a2130e2d2a5e560209952a3dce1ccb8
conn_log_predictions_analysis.png

e44221a0e3d2ffb76b42ca9c341b4936ce0ab0fea754a482424718b4eb8ef660
SS_Scan.pcapng

83d41082f1873ab591dd4cf3bc3a95b1d0fddb7dc7083888edbb5f73d192868b
TC6_Results.png

H. Next Steps / Follow-up Actions:

Proceed with TC7

Execution ID: RUN-20251206-001 (e.g., RUN-20231027-001)

Date: 2025-12-06

Start Time (UTC): 14:02:06

End Time (UTC): 14:09:37

Researcher(s): Teaundra Thompson

A. Linked Documents:

Test Plan: TP-001-v1.5

Test Case(s): TC7

B. Objective:

Determine if the MLP can detect slow fragmented port scanning; **Test case is observatory**

C. Summary of Actions:

Before I began, I made a change to the testbed; I added an additional network adapter "Host-Only" so I could capture traffic from the Kali Linux VM. This test case was run in the same session as TC6. I then powered on the Kali Linux VM and ensured the network adapter was "Host Only". I got the IP address from the Zeek VM by running command **ifconfig**; the IP address was 192.168.56.104. I opened Wireshark on the Zeek VM and started the packet capture on interfaces enp0s3 and enp0s8. I went back to the Kali Linux VM and ran the command **sudo nmap -sS -T1 -f 192.168.56.104** for three minutes. Whilst the command was running I engaged in light web browsing on the Zeek VM. I only made simple google searches and did not click any links. Once the three minutes were over, I saved the pcap as **Slow_Scan.pcapng** in the file path **Desktop/testCases/TC7**. I then ran the commands **cd ..** and **cd TC7** to get out of the previous test case folder and into the current test case folder. Then I ran the command **zeek -r Slow_Scan.pcapng**. The conn.log was generated and the conn.log from TC6 was deleted out of the Colab runtime and the new conn.log was uploaded. The conn.log was then preprocessed and analyzed.

D. Key Observations (Qualitative Data):

Light web browsing generated a lot of QUIC traffic.

E. Anomalies, Errors, and Deviations from Test Plan:

1. Model behavior when exposed to slow fragmented port scanning

    - Flagged 34.8% of the traffic as anomalous

2. Analysis of traffic

    - MLP classified the traffic as normal

    -The next highest prediction was reconnaissance

F. Testbed State Changes:

Added "Host-Only" network adapter to the Zeek VM

G. Data Pointers:

Raw Data Location: /home/tootie/Desktop/testCases/TC7

Data Integrity Checksums (SHA256):

e21a1030ce3918864948266298704ba9cfb094418487687f51280cbdbbe7f56c conn.log

e640a897925002b410ad60f3d5ccb7216cd1b104e72b9bee66bb7fdf501ef5b4 conn_log_predictions_analysis.png

9bb22e4ed1fd884d01f3c3c7bacf3296de3c75bfde62651cf198a75a1322856b
Slow_Scan.pcapng

bb179fa60b973f4c84a29226542031420badf80512a0d1baab0050c5951356cc
TC7_Results.png

H. Next Steps / Follow-up Actions:

Calculate true and false positive rates from all executions.